刘池帅

189-8676-2051 | liuchishuai2022@163.com | 广东深圳

求职意向：人工智能 后台开发工程师 | 3年经验

教育经历

西安电子科技大学	**电子与通信工程**	**硕士**	2019.09 - 2022.07
武汉科技大学	**电子信息工程**	**本科**	2015.09 - 2019.06

工作经历

腾讯科技（深圳）有限公司	**技术工程事业群**	**后台开发**	2022.07 - 2025.08
百度在线网络技术（北京）有限公司	**移动生态事业群**	**后台开发**	2025.11 - 至今

项目经历

百度AI助手MCP在线服务接入 **开发** 2025.11 - 至今

项目背景

为手机百度APP的AI助手提供MCP（Model ContextProtocol）服务层建设，统一接入冬奥会、本地生活等 20+业务方数据服务，支撑AI助手的多场景信息获取与展示能力。

项目难点

- 接入业务方数量多、迭代节奏快，传统逐个开发模式无法满足上线效率要求；
- 各业务方数据结构差异大，前端展示需要个性化适配，人工编写转换逻辑成本高。

技术方案

- 接入流程配置化：抽象通用接入模板，将服务注册、数据源获取、参数适配等环节标准化为配置项，新服务接入仅需填写配置即可上线，无需代码开发与发版；
- 数据转换智能化：基于自定义Skills实现数据结构的自动解析与转换，将业务方原始数据映射为前端指定的渲染数据结构，替代人工编写转换代码。

项目成果

- 接入效率提升 5倍：日均接入能力从1个提升至5个，且全程免代码发版；
- 转换开发效率提升24倍：单个服务的数据解析转换耗时从人工编写2h缩短至AI端到端生成5min。

基于领域微调cdn客户成本分析系统 **负责人** 2025.06 - 2025.07

- **项目背景：**业务团队每月需为客户生成成本报告，但由于CDN成本分摊算法复杂，分析客户跨月成本波动的原因需要数据分析师投入大量时间进行人工数据提取和比对，效率低下且容易出错。本项目旨在开发一个智能问答系统，允许业务人员通过自然语言提问，自动获得专业、准确的成本变化分析，以解放人力。
- **项目难点：**
  - **专业壁垒高：**成本分析涉及大量CDN领域专业术语和独特的分摊逻辑，要求模型具备深厚的领域知识。
  - **问题多样性强：**用户问题范围广（如从“总体成本为何上升”到“某客户成本为何变好/变差”），通用模型难以精准应对所有场景。
  - **交付周期紧：**业务需求迫切，要求在两个月内交付可用原型。
- **技术方案：**
  - 技术选型与模型微调
    - **基座模型选择：**综合对比了多个开源模型，最终选定DeepSeek-r1-Distill-Qwen-32B作为基座模型。因其在代码和数学推理能力上的优势，能更好地理解成本分摊的逻辑计算，同时较小的参数量满足了短期迭代和低成本部署的要求。
    - **微调策略：**采用 LoRA (Low-Rank Adaptation) 技术进行高效微调，极大降低了微调成本并且分析准确度提高了30%。
    - **领域知识注入（关键工作）：**我主导构建了超过2000组的高质量指令微调数据。数据来源包括：①重构历史问答：将分析师与业务人员的过往QA对话进行重构，整理数据约500条。②专业知识语料补充：整理了100条左右的CDN领域的背景知识，包括成本组成、计费原理，客户组成等等。③数据增强：基于真实QA数据使用腾讯元宝仿写语料，通过人工校验后加入训练数据库。
  - **构建分层处理Pipeline以应对复杂查询**
    - **问题分类器：**为解决“问题多样性”难点，首先使用训练模型将用户问题预先划分为“分析问题”、“查询问题”，”非成本分析问题“3个类别。
    - **策略性提示工程：**针对不同问题类别，我设计了不同的精细化Prompt模板，例如：①分析问题，要求Prompt生成待查询场景指定月份的sql，与数据库交互后模拟真实算法场景，将计算过程写入提示词，减少幻觉概率；②查询问题，生成指定的查询sql，将数据整理返回。③非成本分析问题，直接调用大模型生成回答
  - **评估：**
    - 从历史数据中抽取了500个问题组成测试集，进行盲测评估。最终模型分析的准确率（与人工分析结果一致）达到90%，成功达到业务目标。
- 项目成果：
  - **效率提升：**系统上线后，将原本需要1-2天的成本分析工作缩短至分钟级，每月为业务团队节省约3人/天的人力成本。
  - **质量提升：**分析准确率稳定在90%以上，输出格式规范，减少了人为疏漏。

cdn运营平台建设 **负责人** 2024.07 - 2025.07

- **项目背景：**原 CDN 运营平台基于 Python 开发，存在**查询响应慢**（CDN节点状态、设备历史带宽曲线平均 3s）、**故障恢复慢**（Redis主从故障需人工切换）、**部署环境不一致**三大问题，影响 CDN 节点运维效率与运营决策时效性。作为后台开发，主导平台从 Python 迁移至 Golang，并落地高可用技术方案。
- **技术方案：**
  - **Redis 高可用缓存架构**：采用“1 主 2 从 + 3 哨兵”架构存储节点状态、带宽等热点数据。主从复制确保数据冗余，哨兵集群通过每秒 3 次心跳检测主节点健康状态，故障时自动完成从节点选举与切换（耗时 < 10s），替代原有人工操作。结合 Kafka 消费者的主动缓存刷新机制，缓存命中率从 65% 提升至 92%，彻底解决查询慢问题。
  - **容器化部署**：基于 Docker 封装 Golang 服务及依赖（包括独立部署的 Kafka 消费者），编写统一 Dockerfile 确保开发、测试、生产环境镜像一致。结合CI/CD 流水线实现“代码提交→自动构建→环境部署”自动化，部署步骤从 10 + 步简化为 1 步，耗时从 1 小时 / 次降至 10 分钟 / 次，环境兼容问题清零。
  - **Nginx 故障自愈**：在负载均衡层配置主动健康探测，每 5 秒检查后端服务/health接口，连续 3 次失败自动剔除故障实例，恢复后自动加回，服务可用性从 99.5% 提升至 99.95%。
- **项目成果：**
  - 核心查询接口响应从 3s 优化至 50ms，性能提升 96.7%；

- Redis 故障切换从 30 分钟人工操作缩至 10s 内自动完成；
- 部署效率提升 50%，运营效率（如 CDN 节点故障排查）提升 30%+。

个人总结

- 熟悉模型微调原理与算法，掌握领域适配技术
- 掌握Golang/Python/JavaScript多语言开发
- 熟悉MySQL数据库设计与优化策略
- 熟练掌握Redis缓存中间件
- 熟练使用calude code、trea等AIcoding工具
- 积极关注AI发展，熟练使用skill、mcp等扩展插件，持续提高个人效率